

GENERATION & PROMPT DESIGN

Class 4: From Retrieved Chunks to Perfect Answers

Prompt Engineering, Chain Types, Citations & Full Build

WHAT YOU WILL LEARN IN THIS CLASS:

- RAG Prompt Anatomy — The 3 essential parts of every RAG prompt
- System Prompt Engineering — Preventing hallucination with careful rules
- Chain Types — Stuff, Map-Reduce, Refine & Map-Rerank deep dive
- Citation Strategies — Making the LLM cite sources properly every time
- Handling 'I Don't Know' — When context doesn't have the answer
- Full End-to-End Build — Complete HR Chatbot from zero to working demo

C1
Foundation

C2
Indexing

C3
Retrieval

C4
Generation

C5
Advanced

C6
Production

6-Class RAG Mastery Roadmap — You Are Here: Class 4

Class 4 — Table of Contents

- 4.1 Recap & The Big Picture** — *Where generation fits in the RAG pipeline*
- 4.2 The RAG Prompt — 3 Essential Parts** — *System, Context & Question — anatomy explained*
- 4.3 Part 1: System Prompt Engineering** — *The rules that control LLM behavior in RAG*
- 4.4 Part 2: Context Formatting** — *How to structure retrieved chunks for the LLM*
- 4.5 Part 3: Question Framing** — *Guiding the LLM to answer precisely*
- 4.6 Preventing Hallucination** — *7 techniques to keep the LLM grounded*
- 4.7 Handling 'I Don't Know'** — *When the context doesn't contain the answer*
- 4.8 Citation Strategies** — *Making the LLM cite sources every time*
- 4.9 Chain Types Deep Dive** — *Stuff, Map-Reduce, Refine & Map-Rerank*
- 4.10 Choosing the Right LLM** — *GPT-5, Claude 4.6, Gemini 3, Llama 4 compared for RAG*
- 4.11 Temperature & Generation Parameters** — *What to set and why for factual Q&A;*
- 4.12 Full End-to-End Build** — *HR Chatbot — complete working system walkthrough*
- 4.13 Real-World Prompt Examples** — *5 industry prompt templates you can use today*
- 4.14 Common Generation Mistakes** — *Top 8 failures with fixes*
- 4.15 Summary & What's Next** — *Key takeaways + preview of Class 5*

4.1 | Recap & The Big Picture

Where generation fits in the RAG pipeline

We've covered Indexing (Class 2) and Retrieval (Class 3). Now we have the **right chunks** in hand. The final step: turning those chunks into a **clear, accurate, cited answer**. This is where **Generation** happens — the LLM reads the retrieved context and produces the response.

Where We Are

Phase 1: INDEXING ■ (Class 2)

Documents → Chunk → Embed → Store

Phase 2: QUERYING

Step A: RETRIEVAL ■ (Class 3)

Query → Hybrid Search → Rerank → Top-K Chunks

Step B: ★ GENERATION ★ (Today — Class 4)

Top-K Chunks + System Prompt + Question → LLM → Final Answer

Generation is the LAST mile — it converts raw retrieved data into a human-friendly, cited response.

Why Generation Quality Matters

Even with perfect retrieval, bad generation = bad user experience:

- LLM ignores the context and answers from its own memory → HALLUCINATION
- LLM gives a correct answer but doesn't cite the source → NOT TRUSTWORTHY
- LLM dumps all context back verbatim → NOT HELPFUL (user wants a summary)
- LLM says 'I don't know' when the answer IS in the context → FRUSTRATING

The prompt template is the control layer that prevents all of these failures.

4.2 | The RAG Prompt — 3 Essential Parts

System Instructions + Context + Question

Every RAG prompt in the world has exactly **three parts**. Understanding each part and designing it carefully is the difference between a RAG system that works and one that hallucinates.

Part	What It Contains	Purpose	If Missing...
Part 1: System Instructions	Rules for the LLM: what to do, what NOT to do, output format, tone, citation requirements	Controls LLM behavior. Prevents hallucination. Defines the 'personality' of the assistant.	LLM uses its own knowledge instead of context. May hallucinate. No citations.
Part 2: Retrieved Context	The actual text chunks retrieved from the vector database, with metadata (source file, page number)	Provides the KNOWLEDGE the LLM should use to answer. This is the 'open book' in the exam.	LLM has nothing to reference. Will either refuse to answer or hallucinate entirely.
Part 3: User Question	The original natural language question asked by the user	Tells the LLM WHAT to answer. Focuses the response on the specific query.	LLM doesn't know what to do with the context. May summarize everything instead of answering specifically.

Complete Prompt Example

```
// Complete RAG Prompt – All 3 Parts
```

```
■■■ PART 1: SYSTEM INSTRUCTIONS ■■■
```

```
You are AmanCorp's HR assistant. Follow these rules strictly:
```

1. Answer ONLY using the provided context below.
2. If the answer is NOT in the context, say: 'This information is not available in the HR policy. Please contact HR directly.'
3. Always cite the source: [Source: filename, Page: number]
4. Be concise – answer in 2-4 sentences maximum.
5. Use a professional, friendly tone.

```
■■■ PART 2: RETRIEVED CONTEXT ■■■
```

```
[Chunk 1] Source: hr_policy_2024.pdf | Page: 12 | Section: 3.2  
'All full-time employees are entitled to 24 days of annual leave  
per calendar year. Leave must be applied at least 5 working days  
in advance through the HR portal.'
```

```
[Chunk 2] Source: hr_policy_2024.pdf | Page: 13 | Section: 3.5  
'Unused leave can be carried forward for up to 90 days into the  
next calendar year. After 90 days, unused leave expires.'
```

```
■■■ PART 3: USER QUESTION ■■■
```

```
How many leaves do I get and can I carry them forward?
```

```
■■■ LLM GENERATES ■■■
```

```
'You are entitled to 24 days of annual leave per year. Leave must  
be applied 5 days in advance via the HR portal. Unused leave can  
be carried forward for up to 90 days into the next year, after  
which it expires. [Source: hr_policy_2024.pdf, Pages 12-13]'
```

4.3 | Part 1: System Prompt Engineering

The rules that control LLM behavior in RAG

The system prompt is the **most important part** of the RAG prompt. It's the instruction manual that tells the LLM how to behave. A weak system prompt leads to hallucination, missing citations, and off-topic answers. A strong system prompt keeps the LLM grounded and reliable.

The 7 Essential System Prompt Rules

#	Rule	What to Write	Why It Matters
1	Ground in context only	'Answer ONLY using the provided context below. Do not use your own knowledge.'	Prevents hallucination. Forces LLM to use retrieved chunks, not training data.
2	Handle missing info	'If the answer is NOT in the context, say: I don't have this information in the provided documents.'	Prevents the LLM from guessing when the context doesn't contain the answer.
3	Require citations	'Always cite the source: [Source: filename, Page: number] at the end of your answer.'	Enables verification. Users can check the original document.
4	Set response length	'Be concise. Answer in 2-4 sentences unless the question requires more detail.'	Prevents the LLM from dumping entire chunks verbatim. Keeps answers focused.
5	Define tone	'Use a professional, friendly tone. Avoid jargon unless the user uses it first.'	Creates consistent user experience. Matches brand voice.
6	Define role	'You are AmanCorp's HR assistant, specializing in leave policy and employee benefits.'	Focuses the LLM on a specific domain. Reduces off-topic responses.
7	Set boundaries	'Do not answer questions about salary, performance reviews, or personal employee data.'	Prevents the chatbot from addressing sensitive topics it shouldn't handle.

Weak vs Strong System Prompt Comparison

	Weak System Prompt	Strong System Prompt
Prompt	'You are a helpful assistant. Answer questions about HR.'	'You are AmanCorp's HR policy assistant. RULES: 1) Answer ONLY from the provided context. 2) If the answer is not in the context, say: This information is not available in the HR policy. 3) Always cite [Source: file, Page]. 4) Be concise (2-4 sentences). 5) Do not discuss salary or performance data.'

	Weak System Prompt	Strong System Prompt
When asked about leave	Answers correctly (usually) but may add extra info from training data that's wrong for this company	Answers ONLY from retrieved HR policy chunks. Cites exact page number. Sticks to 2-4 sentences.
When asked about something NOT in context	Makes up a plausible-sounding answer (HALLUCINATION)	Responds: 'This information is not available in the HR policy. Please contact HR directly.'
When asked about salary	Might answer with generic salary info from the internet	Responds: 'I'm not able to assist with salary-related queries. Please contact your HR representative.'

4.4 | Part 2: Context Formatting

How to structure retrieved chunks for the LLM

How you present the retrieved chunks to the LLM matters significantly. Poor formatting confuses the model. Good formatting helps it find and use the right information.

Context Formatting Best Practices

Practice	What To Do	Why
Number each chunk	Label as [Chunk 1], [Chunk 2], etc.	LLM can reference specific chunks. Easier to trace which chunk informed the answer.
Include metadata	Show source file, page number, section heading for each chunk	LLM uses this metadata for citations. User can verify the source.
Use clear delimiters	Separate chunks with '---' or XML tags or numbered headers	LLM clearly sees where one chunk ends and another begins. Prevents blending.
Order by relevance	Put the most relevant chunk FIRST, second-most LAST, rest in middle	Addresses the 'lost in the middle' problem — LLMs attend most to beginning and end.
Trim irrelevant parts	If a chunk has clearly irrelevant content, trim it before sending	Reduces noise. LLM focuses on the useful parts. Saves tokens and cost.

Good vs Bad Context Formatting

Bad — No structure, no metadata

```
// Bad: Chunks merged, no source info
```

Context:

All full-time employees are entitled to 24 days of annual leave per calendar year. Leave must be applied at least 5 working days in advance. Unused leave can be carried forward for up to 90 days. Medical leave is 12 days per year with a valid doctor's certificate.

Good — Numbered, with metadata, clearly separated

```
// Good: Numbered, metadata included, clearly separated
```

Context:

[Document 1]

Source: hr_policy_2024.pdf | Page: 12 | Section: 3.2 - Annual Leave

Content: All full-time employees are entitled to 24 days of annual leave per calendar year. Leave must be applied at least 5 working days in advance.

[Document 2]

Source: hr_policy_2024.pdf | Page: 13 | Section: 3.5 - Leave Carry Forward

Content: Unused leave can be carried forward for up to 90 days into the next calendar year. After 90 days, unused leave expires automatically.

4.5 | Part 3: Question Framing

Guiding the LLM to answer precisely

The third part of the prompt is the user's question. While you can pass it as-is, adding a brief instruction before the question guides the LLM to produce a better response.

Approach	Example	When to Use
Direct pass-through	'Question: How many leaves do I get?'	Simple factual questions. Works fine for most cases.
With answer format hint	'Question: How many leaves do I get? Answer in 1-2 sentences with the exact number.'	When you want consistent, concise responses.
With output structure	'Question: Compare maternity and paternity leave. Answer as a comparison with bullet points for each.'	When the user expects a structured/comparative answer.
With step instruction	'Question: How do I apply for leave? Provide step-by-step instructions.'	When the answer should be a process or procedure.
With audience hint	'Question: What is the leave policy? Explain as if to a new employee who just joined.'	When the response should match a specific audience level.

4.6 | Preventing Hallucination

7 techniques to keep the LLM grounded in context

Hallucination is the #1 enemy of RAG systems. Even with perfect retrieval, the LLM may add information from its training data that doesn't exist in your documents. Here are 7 proven techniques:

#	Technique	How It Works	Effectiveness
1	Explicit grounding instruction	System prompt says: 'Answer ONLY from the provided context. Do not add any external knowledge.'	Essential. This one instruction reduces hallucination by 60-70%.
2	Temperature = 0	Set LLM temperature to 0 for deterministic, factual responses. Higher temperature = more creative = more hallucination risk.	Very effective for factual Q&A.; Always use temperature=0 for RAG.
3	'I don't know' fallback	Instruct: 'If the answer is not in the context, respond with a specific fallback phrase instead of guessing.'	Critical. Without this, the LLM will ALWAYS try to answer, even when it should refuse.
4	Citation requirement	Force the LLM to cite which chunk it used: 'Always mention the source document and page number.'	If the LLM can't cite a source, it's likely hallucinating. Citations create accountability.
5	Context-answer alignment check	After generation, verify that every claim in the answer exists in the retrieved context (automated or manual).	Post-generation safeguard. Can be automated with RAGAS Faithfulness metric.
6	Structured output format	Ask the LLM to output JSON with separate 'answer' and 'source_chunks_used' fields. Forces explicit source mapping.	Very effective for programmatic verification. Easy to validate automatically.
7	Chunk relevance threshold	Don't send chunks with similarity score below 0.70. If no chunks pass the threshold, return 'no relevant information found'.	Prevents the LLM from receiving irrelevant context that it might misinterpret.

The Minimum Anti-Hallucination Stack

At bare minimum, EVERY RAG system should use these three:

1. 'Answer ONLY from context' in system prompt
2. Temperature = 0
3. 'I don't know' fallback instruction

These three alone prevent 80%+ of hallucination. Add the other four for production systems.

4.7 | Handling 'I Don't Know'

When the context doesn't contain the answer

One of the hardest problems in RAG: what should the system do when the retrieved chunks don't contain the answer? Without explicit handling, LLMs will **always try to answer** — even if it means hallucinating.

The 3 Scenarios

Scenario	What Happens	Correct Response
Answer IS in context	Retrieved chunks contain the answer to the user's question. This is the happy path.	Generate the answer from context. Cite the source. Normal RAG flow.
Answer is PARTIALLY in context	Some relevant info exists but not the complete answer.	Answer what you can from context. Clearly state what information is missing. Suggest where to find the rest.
Answer is NOT in context	Retrieved chunks are irrelevant or don't address the question at all.	Say: 'I don't have this information in the provided documents.' Suggest alternative resources (HR email, help desk, etc.)

Example Fallback Responses by Industry

Industry	Fallback Response Template
HR Chatbot	'This information is not available in the current HR policy documents. Please contact the HR team at hr@company.com or visit the HR portal for assistance.'
Legal AI	'I could not find relevant information in the indexed case documents. Please consult your supervising attorney or search the full case database manually.'
Customer Support	'I don't have specific information about this in our support documentation. Let me connect you with a live agent who can help. [Transfer to agent button]'
Medical AI	'This query falls outside the scope of the indexed medical guidelines. Please consult the attending physician or refer to the hospital's clinical decision support system.'
E-commerce	'I couldn't find details about this product in our catalog. The product may be discontinued or not yet listed. Please contact customer care for the latest availability.'

4.8 | Citation Strategies

Making the LLM cite sources every time

Citations are what make RAG trustworthy. Without citations, the user has no way to verify the answer — it's just another AI-generated response they can't trust. Good citations include the source document name, page number, and ideally the section heading.

5 Citation Formats

Format	Example	Best For
Inline citation	'You get 24 days annual leave [hr_policy.pdf, p.12].'	Simple chatbot responses. Quick to verify.
End-of-answer citation	'You get 24 days annual leave.\n\nSources:\n- hr_policy.pdf, Page 12, Section 3.2'	Clean answers with sources listed separately. Most professional look.
Numbered references	'You get 24 days annual leave [1].\n\n[1] hr_policy.pdf, Page 12'	Academic/research style. Good for multi-source answers.
Clickable links	'You get 24 days [View source](/docs/hr_policy.pdf#page=12).'	Web apps where users can click to see the original document.
JSON structured	'{"answer": "24 days", "sources": [{"file": "hr.pdf", "page": 12}]}'	API responses. Frontend can render citations however it wants.

How to Force Citations in System Prompt

Add this to your system prompt:

'CITATION RULES:

- Every factual claim **MUST** include a citation.
- Format: [Source: filename, Page: number]
- If you cannot cite a source for a claim, **DO NOT** include that claim.
- If multiple sources support a claim, cite all of them.'

This instruction alone forces the LLM to cite sources 90%+ of the time.

4.9 | Chain Types Deep Dive

Stuff, Map-Reduce, Refine & Map-Rerank

When you have multiple retrieved chunks, how should the LLM process them? There are four standard approaches, each suited for different scenarios:

Chain Type 1: Stuff (Default — Most Common)

Aspect	Details
How it works	ALL retrieved chunks are combined into a single prompt and sent to the LLM in ONE API call. The LLM reads everything at once and generates one answer.
Analogy	Reading all the reference notes on a single page, then writing the answer.
When to use	Most use cases. When k is small (3-6 chunks) and chunks fit within the LLM context window.
Strengths	Fast (single API call). Simple to implement. LLM sees all context at once — can synthesize across chunks.
Weaknesses	Fails if total context exceeds LLM's context window. With many large chunks, you'll hit token limits.
Cost	Low — 1 API call.

Chain Type 2: Map-Reduce

Aspect	Details
How it works	MAP phase: LLM processes each chunk separately and generates a partial answer from each. REDUCE phase: All partial answers are combined and sent to the LLM to generate the final merged answer.
Analogy	5 team members each read one chapter and write a summary. Then the team lead reads all 5 summaries and writes the final report.
When to use	When you have MANY chunks (10-20+) that won't fit in a single prompt. Long documents, multi-document Q&A.;
Strengths	Can handle unlimited chunks. Each chunk gets full attention. Good for comprehensive answers.
Weaknesses	SLOW — requires N+1 API calls (one per chunk + one for reduce). Expensive. May lose cross-chunk connections.
Cost	High — N+1 API calls. If k=10, that's 11 LLM calls per query.

Chain Type 3: Refine

Aspect	Details
How it works	LLM generates an initial answer from chunk 1. Then reads chunk 2 and REFINES the answer. Then reads chunk 3 and refines again. Iterative improvement through each chunk.

Aspect	Details
Analogy	Writing an essay draft, then improving it after reading each new reference one at a time.
When to use	When answer quality is paramount. Sequential documents (like a story or process that builds on itself).
Strengths	Highest quality answers. Each iteration can add nuance and correct earlier mistakes.
Weaknesses	SLOWEST method — N sequential API calls (can't parallelize). Very expensive.
Cost	Very high — N sequential API calls. Can't be parallelized.

Chain Type 4: Map-Rerank

Aspect	Details
How it works	LLM processes each chunk separately and generates an answer + a confidence score (e.g., 0.95). The answer with the HIGHEST confidence score is returned as the final answer.
Analogy	5 team members each answer the question independently. The most confident answer wins.
When to use	Factual lookup questions where ONE chunk contains the complete answer. 'What is the return policy?'
Strengths	Fast (parallel API calls). Good when the answer lives in a single chunk.
Weaknesses	Bad for questions requiring synthesis across multiple chunks. Ignores complementary information.
Cost	Medium — N parallel API calls.

Quick Decision Guide

Scenario	Best Chain Type	Why
k=3-6 small chunks, simple Q&A;	Stuff (default)	Fast, cheap, LLM sees everything at once. Works 90% of the time.
k=10-20 chunks, comprehensive answer needed	Map-Reduce	Handles many chunks by processing each separately then combining.
Maximum quality, sequential documents	Refine	Iterative improvement produces highest quality, but slowest and most expensive.
One chunk has the full answer, fact lookup	Map-Rerank	Pick the single best answer from multiple chunks.

Start with Stuff. Switch to others ONLY when Stuff fails due to context window limits.

4.10 | Choosing the Right LLM for RAG

GPT-5, Claude 4.6, Gemini 3, Llama 4 — compared for RAG

Model	Provider	Context Window	RAG Quality	Cost	Best For
GPT-5.5	OpenAI	256K tokens	★★★★★	\$\$\$	Highest quality answers. Best instruction following.
GPT-4o-mini	OpenAI	128K tokens	★★★★■	\$	Best cost/quality balance. Production workhorse. 10x cheaper than GPT-5.
Claude 4.6 Opus	Anthropic	200K tokens	★★★★★	\$\$\$	Excellent at following complex system prompts. Great for long-context RAG.
Claude 4.6 Sonnet	Anthropic	200K tokens	★★★★■	\$\$	Good quality at moderate cost. Strong at citations.
Gemini 3.1 Pro	Google	2M tokens	★★★★■	\$\$	Massive context window (2M tokens). For huge documents in single prompt.
Llama 4 (70B)	Meta (OSS)	128K tokens	★★★★■	Free (local)	Best open-source option. Run on your own GPU. Zero API cost.
Llama 4 (8B)	Meta (OSS)	128K tokens	★★★██	Free (local)	Lightweight. Runs on consumer GPU. Good for prototyping.
Mistral Large 2	Mistral	128K tokens	★★★★■	\$\$	Strong European alternative. Good multilingual support.

Recommendation for Most RAG Projects (2026)

Prototyping / Learning → GPT-4o-mini (cheap, good quality, easy API)
Production (managed) → GPT-4o-mini for simple Q&A;, GPT-5.5 or Claude 4.6 Opus for complex
Production (self-hosted) → Llama 4 70B on your own GPUs (zero API cost)
Massive documents → Gemini 3.1 Pro (2M context window = stuff entire books)

The LLM choice matters LESS than you think. Good retrieval + good prompts matter MORE.

4.11 | Temperature & Generation Parameters

What to set and why for factual Q&A;

Parameter	What It Controls	RAG Recommendation	Why
temperature	Randomness of token selection. 0 = deterministic (same input → same output). 1 = creative/random.	0.0 for factual Q&A;	RAG needs consistent, factual answers. Creativity introduces hallucination risk.
top_p	Nucleus sampling. Controls the probability mass considered. Lower = fewer token options.	1.0 (default)	Leave at default when temperature is 0. They interact — changing both can cause issues.
max_tokens	Maximum length of the generated response.	500-1000 tokens	Long enough for a complete answer with citations. Short enough to prevent rambling.
frequency_penalty	Penalizes tokens that appear frequently in the output. Reduces repetition.	0.0 (default)	Leave at default. RAG answers are short enough that repetition isn't a problem.
presence_penalty	Penalizes tokens that have appeared at all. Encourages topic diversity.	0.0 (default)	Leave at default. We WANT the LLM to stay on topic, not diversify.

The Only Setting You Need to Change

For 95% of RAG applications, just set temperature=0.0 and leave everything else at default. That's it. Don't overthink generation parameters — your effort is better spent on improving retrieval quality and system prompt design.

4.12 | Full End-to-End Build

HR Chatbot — complete working system

What We're Building

A complete HR Policy Chatbot for AmanCorp:

- Input: 200-page HR Policy PDF
- Indexed with: ChromaDB + OpenAI Embeddings (from Class 2)
- Retrieval: Hybrid search with k=4 (from Class 3)
- Generation: GPT-4o-mini with anti-hallucination prompt (today)
- Output: Cited, concise answers or 'I don't know' fallback

The Complete Prompt Template

```
// Production RAG Prompt Template

SYSTEM:
You are AmanCorp's official HR Policy Assistant.

RULES (follow strictly):
1. Answer ONLY using the context provided below.
2. Do NOT use your own knowledge or make assumptions.
3. If the answer is not in the context, respond exactly:
   'This information is not available in the HR policy.
   Please contact HR at hr@amancorp.com.'
4. Always cite your source: [Source: filename, Page: number]
5. Be concise: 2-4 sentences maximum.
6. Use professional, friendly language.
7. Do NOT discuss salary, performance, or personal data.

CONTEXT:
{retrieved_chunks}

QUESTION: {user_question}

ANSWER:
```

3 Demo Queries — Showing All Scenarios

Query 1: Answer IS in context (Happy Path)

User

How many annual leaves do I get?

Bot Response

You are entitled to 24 days of annual leave per calendar year. Leave must be applied at least 5 working days in advance through the HR portal. [Source: hr_policy_2024.pdf, Page 12, Section 3.2]

Query 2: Answer NOT in context (Fallback)

User

What is the current stock price of AmanCorp?

Bot Response

This information is not available in the HR policy documents. Please contact HR at hr@amancorp.com or check the finance department portal for stock-related queries.

Query 3: Sensitive topic (Boundary)

User

What is my manager's salary?

Bot Response

I'm not able to assist with salary-related queries as per my guidelines. Please contact your HR representative directly for compensation-related questions.

Notice: All 3 scenarios handled correctly — answer with citation, honest fallback, and boundary enforcement.

4.13 | Real-World Prompt Examples

5 industry templates you can use today

Industry	System Prompt Template	Key Rules
HR Chatbot	'You are [Company] HR assistant. Answer ONLY from context. If unsure, direct to hr@company.com. Cite [Source: file, Page]. No salary/performance discussion.'	1. Context-only 2. Specific fallback email 3. Topic boundaries
Legal AI	'You are a legal research assistant. Provide analysis ONLY from indexed case law and statutes. Always cite case name, year, and relevant section. State clearly when analysis is incomplete. This is NOT legal advice.'	1. Heavy citation 2. Disclaimer 3. Completeness indicator
Customer Support	'You are [Brand] support agent. Answer from product documentation. If product issue, provide troubleshooting steps. If not in docs, say: Let me connect you with a specialist. Tone: helpful, patient, empathetic.'	1. Troubleshooting format 2. Escalation path 3. Brand tone
Medical AI	'You are a clinical reference assistant. Answer ONLY from indexed medical guidelines. Always cite guideline name, section, and evidence level. NEVER provide a diagnosis. Add disclaimer: Consult your physician.'	1. Evidence level citation 2. No diagnosis 3. Mandatory disclaimer
E-commerce Product Q&A;	'You are [Store] product expert. Answer from product specs and reviews. Include product name, price, and key specs. If product not in catalog, say: This product may not be available. Tone: enthusiastic but honest.'	1. Structured product info 2. Availability check 3. Balanced tone

4.14 | Common Generation Mistakes

Top 8 failures with exact fixes

#	Mistake	What Goes Wrong	Fix
1	No grounding instruction in prompt	LLM uses its own training data instead of retrieved context. Answers sound correct but may be wrong for YOUR data.	Add: 'Answer ONLY from the provided context. Do not use external knowledge.'
2	Temperature too high	LLM gives different answers to the same question. Introduces creative but wrong variations.	Set temperature=0.0 for all factual RAG Q&A.;
3	No 'I don't know' fallback	LLM always tries to answer, even when context has no relevant info. Results in confident hallucination.	Add explicit fallback: 'If not in context, say: I don't have this information.'
4	No citation requirement	Answers have no source reference. Users can't verify. Trust is low.	Add: 'Always cite [Source: filename, Page: number] at the end.'
5	Context too long / too many chunks	LLM gets overwhelmed by noise. Ignores relevant info buried in the middle ('lost in the middle' problem).	Reduce k to 3-5. Use reranking. Put best chunk first, second-best last.
6	No topic boundaries	Chatbot answers sensitive questions (salary, personal data) it shouldn't touch.	Add boundary rules: 'Do NOT discuss salary, performance reviews, or personal data.'
7	Using Stuff chain with too many chunks	Total token count exceeds LLM context window. API call fails or truncates.	Switch to Map-Reduce chain for k>8 or total context >80% of context window.
8	Not testing edge cases	System works for happy path but breaks on ambiguous, multi-part, or unanswerable questions.	Test 50+ queries covering: happy path, partial answer, no answer, boundary, multi-part, ambiguous.

4.15 | Summary & What's Next

Key takeaways + preview of Class 5

Class 4 — Key Takeaways

- **RAG prompt has 3 parts:** System Instructions (behavior rules) + Context (retrieved chunks) + Question (user query).
- **System prompt is the most important part** — 7 essential rules: ground in context, 'I don't know' fallback, citations, conciseness, tone, role, boundaries.
- **Context formatting matters:** Number chunks, include metadata, use delimiters, order by relevance (best first, second-best last).
- **Preventing hallucination:** Context-only instruction + temperature=0 + 'I don't know' fallback = 80% of the battle won.
- **Citations build trust:** Force citations in system prompt. Every claim must reference a source document and page.
- **4 chain types:** Stuff (default, fast), Map-Reduce (many chunks), Refine (highest quality), Map-Rerank (fact lookup).
- **Stuff chain works for 90% of cases.** Only switch when context exceeds the LLM's window.
- **LLM choice matters less than retrieval + prompts.** GPT-4o-mini is fine for most RAG. Focus effort on retrieval quality.
- **Temperature = 0.0** for all factual RAG. No exceptions.
- **Test all 3 scenarios:** Answer in context, answer NOT in context, and sensitive/boundary topics.

Preview of Class 5: Advanced RAG Patterns

What We'll Cover in Class 5

Now that you have a complete working RAG system, how do you make it BETTER? Class 5 covers:

- Why Naive RAG Fails for 30-40% of Queries — specific failure analysis
- HyDE (Hypothetical Document Embedding) — smarter query embedding
- Self-RAG — LLM decides when to retrieve and self-critiques
- CRAG (Corrective RAG) — web search fallback when context is weak
- GraphRAG — knowledge graphs + RAG for multi-hop reasoning
- Agentic RAG — LLM agents that plan, retrieve, and verify
- Multi-Modal RAG — images + text retrieval and generation

Class 5 takes you from intermediate to advanced RAG practitioner!

6-Class Roadmap

Class	Title	Status
Class 1	Foundation — What, Why & How RAG Works	■ Completed

Class	Title	Status
Class 2	Indexing Pipeline — Chunking, Embeddings, Vector DB	■ Completed
Class 3	Retrieval — Dense, Sparse, Hybrid, Reranking, MMR	■ Completed
Class 4 (TODAY)	Generation — Prompt Design, Chains, Citations, Full Build	■ Completed
Class 5	Advanced RAG — HyDE, CRAG, Self-RAG, GraphRAG, Agentic	■ Next
Class 6	Production & Interview Prep — RAGAS, Deployment, 20 Q&A;	■ Coming

RAG Mastery — Class 4 of 6 | Created by Aman for AmanAI Lab | amanailab.com